Author Name

February 10, 2026

Let's SCIP it!

SCIP as a Black-Box Solver

- SCIP is a solver for finding globally optimal solutions of
  - mixed-integer (nonlinear) programs
  - satisfiability problems
  - constraint integer programs
- supports exact arithmetic for mixed-integer linear programs

Interfaces

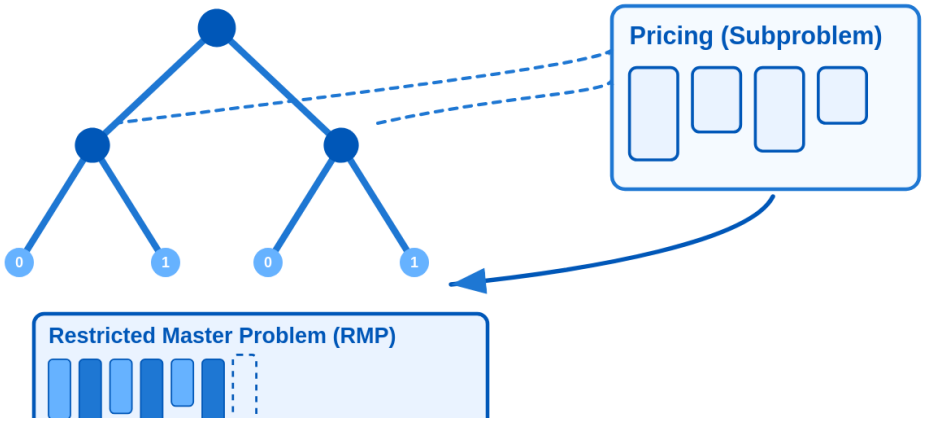
SCIP has interfaces to many popular programming languages

- C/C++
- Python
- Julia
- Rust
- Matlab
- Java

SCIP is Easy to Use

- executable can read many popular file formats (mps, lp, etc.)
- modeling problems via SCIP's interfaces is very easy

Add some Python code here.



Branch-and-Price

1 Out-of-the-box Solver
2 Branch-and-Price
3 Custom Bounds
4 Separation
5 Propagation
6 Presolving
7 Heuristics
8 Branching Rules
9 Benders Decomposition
10 Custom Constraints

Branch-and-Price

required functionality

- column generation
- embedding of column generation in branch-and-bound
- custom branching rules

SCIP naturally supports these features by

- plugin for generating new columns (`pricer`)
- automatic handling of branch-and-bound tree including warm start and heuristics
- plugin for custom branching rules (`branching rule`)

Custom Bounds

required functionality

- access to variables with local bounds (modified during solving process)
- embedding of custom bound in branch-and-bound

SCIP naturally supports these features by

- plugin for computing custom bound (`relaxator`)
- automatic embedding in branch-and-bound

Cutting Planes

required functionality

- access to solution
- creating and adding custom cutting planes

SCIP naturally supports these featurs by

- plugin for custom cutting planes (`separator`)
- automatic embedding in branch-and-bound, including cut storage and selection
- full control of timing of cut generation
  - only at root node
  - every k-th level of the branch-and-bound tree

Variable Bound Propagation

required functionality

- access to local variable bounds (during solving process)
- possibility to change local variable bounds

SCIP naturally supports these featurs by

- plugin for propagation methods (`propagator`)
- automatic embedding in branch-and-bound
- full control of timing of propagation
  - only at root node
  - every k-th level of the branch-and-bound tree
  - before or after solving LP relaxation

Presolving

required functionality

- custom rules for simplifying a problem

SCIP naturally supports these featurs by

- plugin for presolving methods (`presolver`)
- synchronization with other presolving routines
- full control of presolving behavior
  - maximum number of rounds
  - classification of expected required time (fast, medium, exhaustive)

Heuristics

required functionality

- access to local or global variable bounds
- embedding of heuristic search within branch-and-bound

SCIP naturally supports these featurs by

- plugin for heuristics (`heuristic`)
- automatically called during solving process
- automatic check whether solution is feasible
- full control of behavior
  - frequency of calling heuristic
  - before/after solving LP, before/during/after presolving

Branching Rules

required functionality

- access to LP solution and local variable bounds
- add branching decision

SCIP naturally supports these featurs by

- plugin for branching rules (`branching rule`)
- automatic embedding in branch-and-bound
- supports
  - variable branching
  - constraint-based branching

Benders Decomposition

required functionality

- access to LP or IP solution
- embedding of Benders cuts in branch-and-bound

SCIP naturally supports these featurs by

- plugin for custom Benders decomposition (`benders`)
- plugin for custom Benders cuts (`benderscut`)
- automatic embedding of Benders cuts into branch-and-bound

Custom Constraints

required functionality

- abstract mechanism to define constraints (not necessarily represented by linear inequalities)
- accept or reject solutions

SCIP naturally supports these featurs by

- plugin for custom constraints (`constraint handler`)
- features
  - mechanism to accept or reject (heuristic) solutions
  - presolving reductions
  - cutting plane separation
  - propagation

You wonder how to solve mixed-integer (nonlinear) programs?

You wonder how to solve mixed-integer (nonlinear) programs?

Let's SCIP it!